

tie 语言语法规范

tie Language Specification

版本 2026

第 1 章	词法与记号
第 2 章	类型系统
第 3 章	运算符与表达式
第 4 章	语句
第 5 章	函数
第 6 章	数据结构
第 7 章	接口
第 8 章	错误处理
第 9 章	模块与导入
第 10 章	并发
第 11 章	不安全机制
第 12 章	宏与元编程
第 13 章	所有权与移动
第 14 章	界面元素块与图语法
第 15 章	工具链与语言服务
第 16 章	速查表

第 1 章 词法与记号

本章定义一个 tie 源文件的物理形态：如何组织、如何切分为记号、哪些记号有意义。词法层是其余一切章节的基础——语法分析只看到本章定义的记号流，而不关心源文件的字节细节。

1.1 源文件结构：头部与内容

一个 tie 源文件由两部分组成：头部声明本文件是什么，内容承载真正的代码或数据。两者职责分离，头部只占最前面几行。

```
type tie<logic>          // 头部：声明文件角色
                        // （其后为内容区）

func main() {
    println("hello")
}
```

头部是真正的语法行，而不再是以注释形式书写的编译器指令。头部必须位于文件最前若干行，连续排列（其间允许空行），直到第一个非头部行为止。

头部语法

头部由关键字 `type`、语言名 `tie` 与可选的尖括号子类型构成：

- `type tie<logic>` —— 带子类型的完整形式
- `type tie` —— 裸形式，表示 `Type` 角色本身

子类型写在尖括号内，只允许写一个。若文件未声明头部，按 `logic` 处理。

文件名与角色的一致性

文件名可以采用 `xxx.角色.tie` 的形式声明默认角色。当文件名与头部声明不一致时，以头部为准，并给出一条警告。这样既允许用文件名快速辨别用途，又不让文件名凌驾于显式声明之上。

1.2 文件角色

角色决定编译器如何解析与生成该文件。角色是文件的第一属性，它决定了后续内容允许出现什么构造。

头部	角色与允许的内容
<code>type tie<logic></code>	逻辑代码。可含函数、结构、模块级声明；可编译为可执行程序或静态库。默认角色。
<code>type tie<class></code>	类库文件。与 <code>logic</code> 同为逻辑代码，但只编译为静态库，不产生入口。
<code>type tie<script></code>	脚本。顶层语句按书写顺序直接执行，不要求显式入口。
<code>type tie<data></code>	纯数据。只允许数据声明与字面量，不允许函数体；可被其他文件导入作为常量源。
<code>type tie<ui></code>	界面文件。声明界面元素树与事件处理函数，由编译器生成界面代码。
<code>type tie<db></code>	数据库文件。声明表结构，编译器生成建表与读写代码。
<code>type tie<port></code>	接口文件。声明对外契约（方法签名集合），不含实现。
<code>type tie<ir></code>	中间表示文件。直接产出中间代码，不再经过后续优化与链接阶段。

角色不是注释、不是标记，而是编译器分派解析与生成策略的依据。这正是头部必须在最前面且被词法层正式识别的原因。

1.3 字符集与编码

源文件以 UTF-8 编码。字符串内部同样以 UTF-8 存储，但语言层面区分两个尺度：

- `str_len(s)` / `str_char(s, i)` —— 以码点为单位。中文、emoji 都算一个。
- `len(s)` —— 以字节为单位。一个汉字占 3 字节。

遍历文本一律使用码点尺度；按字节遍历会把多字节字符切碎。这一约定贯穿标准库的文本处理函数。

```
var s = "你好"
str_len(s)    // 2   码点
len(s)       // 6   字节
str_char(s, 0) // "你"
```

1.4 注释

注释有两种形态，二者在词法阶段被丢弃，不进入记号流。

```
// 行注释：自 // 起至本行结束
/* 块注释：可跨行 */
```

文档注释以三条斜线书写，绑定到紧随其后的声明上，可被工具提取。

```
/// 计算两数之和。
/// 文档注释绑定到下面的函数。
func add(a: i64, b: i64) -> i64 { return a + b }
```

1.5 标识符

标识符由字母、下划线或 Unicode 字母开头，其后可接字母、数字与下划线。语言同时接受 Unicode 标识符，中文可以直接用作标识符。

```
count, _tmp, 用户数, 结果1
```

约定：类型名以大写字母开头，函数与变量以小写字母或下划线开头。这是约定而非强制，编译器不做检查，但工具链会据此提供提示。

1.6 关键字

关键字是保留字，不可用作标识符。它们按用途分组如下。

用途	关键字
声明	type, var, const, func, struct, namespace, port, impl, import, as, pub, unsafe, extern, macro
类型	i8 i16 i32 i64 u8 u16 u32 u64 i128 u128 f32 f64 bool trit char string void code table map num text misc any self
语句	if else while for in switch case default break continue return yield goto
修饰	ref immut move inline retry with when extends alias
字面量	true false zero
并发与运行	actor async spawn await panic

关键字表是封闭的：任何新关键字都只在语言版本升级时加入。语言预留了少量保留词，它们当前没有语义，但同样不可用作标识符，以便未来扩展时不必破坏既有代码。

1.7 数值字面量

整数支持四种进制，允许用下划线分组以提高可读性。小数与指数形式构成浮点字面量。

形态	含义
42	十进制整数，默认类型 i64
0xFF / 0xff	十六进制
0b1011	二进制
0o755	八进制
1_000_000	下划线分组，等价于 1000000
3.14 / 1.5e3	浮点，默认类型 f64
1.2f32	带后缀，显式指定为 f32

1.8 布尔与三值字面量

true 与 false 构成 bool。语言另有一个三值类型 trit，取值为正、零、负，其字面量复用 true / zero / false，在 trit 上下文中适配为三值。

```
var b: bool = true
var p: trit = true    // 正
var z: trit = zero    // 零
var n: trit = false   // 负
```

1.9 字符与字符串字面量

字符字面量以单引号界定，字符串字面量以双引号界定。二者都支持转义序列。

```
var c: char = 'A'
var s: string = "hello"
var esc: string = "tab:\t quote:\" backslash:\\ newline:\n"
```

字符串可用花括号插入表达式，插入的内容会被转换为文本后拼入。

```
var user = "tie"
var msg = "hello, {user}! n={str_len(user)}"
```

在需要字面花括号的场合（例如书写 JSON 片段），用字符构造函数拼接，或先将该片段存入变量，避免与插值语法冲突。

1.10 自动分号

`tie` 不要求每条语句以分号结尾。编译器在词法阶段按规则补全语句结束，这一机制称为自动分号插入。

- 当一行结束后，若下一个记号不能延续当前语句，则在此处插入语句结束。
- 当一行以逗号、二元运算符、左括号等「明显未结束」的记号结尾时，不插入。
- 显式书写分号始终合法，可与自动补全混用。

这条规则让代码更接近日常书写的文本，同时保持分析的确定性——判定只看记号，不依赖缩进或换行风格。

1.11 运算符记号

下表汇总全部运算符记号。含义与结合性见第 3 章。

记号	用途
<code>+ - * / %</code>	算术
<code>** **?</code>	幂; <code>**?</code> 为带检查的幂
<code>== != < > <= >=</code>	比较
<code>&& !</code>	逻辑 (对 <code>trit</code> 为三值逻辑)
<code>& ^ << >> ~</code>	位运算
<code>= += -= *= /= %=</code>	赋值与复合赋值
<code>%%</code>	原地修改 (见第 3 章)
<code>=></code>	数据流管道; 在函数值位置为短闭包
<code>in not in</code>	成员判定
<code>?.</code>	安全默认值
<code>? ??</code>	存在判定与空值合并
<code>\</code>	解引用
<code>.. ..=</code>	范围 (左闭右开 / 左闭右闭)
<code>:</code> <code>::</code> <code>.</code> <code>,</code> <code>;</code>	分隔与路径
<code>@ # \$</code>	元素块、编译期指令、宏插值

第 2 章 类型系统

`tie` 是静态类型语言: 每个表达式在编译期有确定的类型, 类型错误在编译期报告, 不进入运行期。类型系统同时承担两件事——描述数据的形状, 以及表达访问权限 (谁可以写、谁只能读、谁拥有所有权)。后者见第 5 章与第 13 章。

类型分为若干族: 基本类型、复合类型 (表、元组、映射)、聚合类型 (结构、枚举)、函数与接口类型、以及两种特殊类型 (宽类型与动态类型)。

2.1 基本类型

基本类型直接对应机器表示, 没有隐藏的分配与间接层。

类型	含义
i8 i16 i32 i64	有符号整数，宽度以位计
u8 u16 u32 u64	无符号整数
i128 u128	128 位整数
f32 f64	浮点数
bool	布尔，取值 true / false
trit	平衡三进制，取值正 / 零 / 负（见 2.3）
char	单个字符，按码点计
string	不可变文本，UTF-8 存储
void	无值，仅作函数返回类型
code	编译期代码片段，无运行期实体（见第 12 章）

整数默认类型为 i64，浮点默认类型为 f64。字面量在需要的上下文中会按目标类型调整。

2.2 平衡三进制 trit

trit 是一个三值类型，取值正、零、负。它用于三路比较、三值逻辑与平衡三进制数论。与布尔相比，它多了一个「未知」状态，因此比两者择一更贴近真实判定。

```
var cmp: trit = compare(a, b) // 正 表示 a>b, 零 表示相等, 负 表示 a<b
if cmp == zero {
    // 相等分支
}
```

三值逻辑的运算规则与布尔一致地推广：与运算取小、或运算取大、非运算取反。混合 trit 与整数的运算按提升规则转为整数，除法与取余对 trit 不开放。

2.3 宽类型

宽类型是一组类型的类别名，用于在书写时省略具体宽度，或在多种数值类型间统一处理。它只存在于编译期：变量在参与运算时使用推导出的具体类型。

关键词	覆盖范围
num	全部数值类型：整数与浮点
text	文本类型：string 与 char
misc	其余类型：bool、void、code 等

```
var a: num = 42      // 推导为 i64
var b: num = 3.14    // 推导为 f64
var s: text = "hi"   // 推导为 string
```

宽类型最常见的用途是编写同时接受多种数值宽度的函数，例如统计与格式化工具。

2.4 表类型

表是动态长度的同质序列，元素类型写在尖括号内。表有两种创建方式：字面量表（长度固定，元素在书写时确定）与动态表（可增长，用构造器创建）。

```
var fixed: table<i64> = [1, 2, 3]           // 字面量表
var dyn = table_new_i64()                  // 动态表，先用后增
table_push(dyn, 1)
table_push(dyn, 2)
table_push(dyn, 30)
```

表可嵌套，形成多维结构。遍历使用 `for-in`，长度用 `len`。

```
var grid: table<table<i64>> = [[1, 2], [3, 4]]
for row in grid {
    for cell in row {
        println(to_string(cell))
    }
}
```

2.5 映射类型

映射是以键索引的集合，键与值都有类型。键可以是文本、整数或枚举。

```
var age: map<i64> = ["alice": 3, "bob": 5]
age["carol"] = 7
if age.has("alice") {
    println(to_string(age["alice"]))
}
```

映射的字面量语法与表一致，区别在于元素写成「键：值」。当键为字符串时，可用点号访问其值，这在表达结构化数据时更易读。

2.6 元组类型

元组把若干个类型组合成一个值，位置固定、类型可不同。元组用于返回多个结果，或表达「这几个值一起才有意义」的场合。

```
func min_max(xs: table<i64>) -> (lo: i64, hi: i64) {
    // ...
    return 1, 9
}

var lo, hi = min_max([3, 1, 9])
```

元组可命名分量，命名同时作为文档与解构依据。也可不命名，此时用位置访问。

2.7 类型标注与推导

变量声明可以写类型，也可以省略。省略时编译器从初始化表达式推导类型。

```
var n: i64 = 0          // 显式标注
var m = 0              // 推导为 i64
var s = "text"         // 推导为 string
```

类型标注在需要跨模块契约、或者推导结果与意图不符时使用。函数签名必须写参数与返回类型——接口边界上的类型不做推导。

2.8 泛型

泛型让类型与算法解耦。类型参数写在尖括号中，编译期为每个具体类型生成一份实现，因此运行期没有额外开销，也没有虚表跳转。

```
func max_of<T>(a: T, b: T) -> T {
    if a > b { return a }
    return b
}

struct Box<T> {
    var value: T
}
```

类型参数可带约束，约束由接口描述（见第 7 章）。约束在编译期检查，不满足即报错。

```
func render_all<T: Drawable>(items: table<T>) {
    for it in items {
        it.draw(0)
    }
}
```

2.9 枚举

枚举是标签联合：一个值要么是这个标签、要么是那个标签，并可按标签携带数据。它用于表达「几种情况之一」的数据，比用整数代号更安全——每个分支都被检查。

```
enum Shape {
    Circle(r: f64)
    Rect(w: f64, h: f64)
}

func area(s: Shape) -> f64 {
    switch s {
        case Shape.Circle(r):
            return 3.14159 * r * r
        case Shape.Rect(w, h):
            return w * h
    }
}
```

枚举也可以做通用型参数，用于表达可能失败或可能缺失的结果，这是第 8 章错误处理的基础。

2.10 动态类型

`any` 表示「任意类型」。它在需要容纳异质数据的场合使用，例如配置表、序列化中间结果。任何值都可以赋给 `any`，取用时需要显式转换。

```
var v: any = 42
v = "text"
if is_num(v) {
    var n = as_i64(v)
}
```

静态类型是默认选择。`any` 把类型错误的发现从编译期推迟到运行期，因此只在确实需要异质容器时使用，而不是作为绕过类型检查的手段。

2.11 类型转换

数值类型之间的转换是显式的。窄化转换可能丢失信息，语言要求书写者明确表达意图。

```
var a: i64 = 1000
var b = to_i32(a)      // 窄化：显式
var c: f64 = to_f64(a) // 整数到浮点
var d = to_string(a)   // 到文本
```

指针与整数之间的转换只在 `unsafe` 上下文中允许（见第 11 章）。聚合类型与动态类型之间的转换由各自的构造与匹配机制承担。

第 3 章 运算符与表达式

表达式由运算符与操作数构成。本章按优先级自高到低说明各运算符的形态与语义，并单列 `tie` 特有的若干记号——它们的共同目标是让常见意图有专有的书写形式，而不是靠函数调用或状态变量绕行。

3.1 优先级与结合性

优先级自高到低如下表。同一行的运算符优先级相同，按结合性方向结合。括号可覆盖默认优先级。

级	运算符	结合性
1	() [] . :: ?.	左
2	! ~ \ 一元 -	右
3	** **?	右
4	* / %	左
5	+ -	左
6	=	左
7	<< >>	左
8	&	左
9	^	左
10		左
11	== != < > <= >= in not in	左
12	&&	左
13		左
14	?:	右
15	->	左
16	%%	左

3.2 算术运算

算术运算符对整数与浮点生效，规则与通常预期一致。整数除零在运行期报错，语言不做静默截断。

```
var a = 7 + 3      // 10
var b = 7 / 3      // 2    整数除法
var c = 7.0 / 3.0  // 2.333 浮点除法
var d = 7 % 3      // 1    取余
```

一元负号改变符号。整数运算在溢出时按位宽回绕；需要检测溢出的场合，使用带检查的运算形式（见 3.12）。

3.3 幂运算

幂运算写作两个星号，右结合。它与乘法的优先级关系使得 `2 ** 3 ** 2` 解析为 `2 ** (3 ** 2)`。

```
var p = 2 ** 10      // 1024
var q = 2 ** 3 ** 2  // 512  右结合
```

带检查的幂写作 `**?`，在结果超出该类型的表示范围时报告错误，而不是回绕。数值敏感的计算应使用这一形式。

3.4 比较运算

比较运算符返回 `bool`。对文本按码点顺序比较；对元组与表按元素依次比较。

```
1 == 1          // true
"abc" < "abd"   // true
```

相等比较要求两侧类型相容。语言不为不同类型之间定义隐式相等——这避免了整数与浮点混比时的意外结果。

3.5 逻辑运算与三值逻辑

逻辑运算符对 `bool` 生效，并短路求值：与运算在左操作数为假时不再求右操作数，或运算在左操作数为真时不再求右操作数。

当操作数是 `trit` 时，同一组运算符按三值逻辑解释：与取较小值、或取较大值、非运算取反。由于三值包含「未知」，与运算中只要有否即为否，或运算中只要有真即为真，未知只在无法决定时才保留。

```
var t: trit = unknown_result()
if t && positive_signal() {
    // 仅当两者皆正
}
```

3.6 位运算

位运算符作用于整数，移位运算的右操作数是移位数。

```
var m = 0b1100 & 0b1010 // 0b1000
var n = 1 << 4           // 16
var x = ~0               // 按位取反
```

3.7 成员判定

成员判定用于回答「这个键在不在这个结构里」。它是布尔表达式的一部分，可直接用于条件。

```
if "alice" in age {
    println("已知")
}
if "dave" not in age {
    println("未见")
}
```

判定对象可以是映射（判键）、表（判元素）、文本（判子串）。取反形式 `not in` 是一个整体记号，不写成 `!(x in y)`——后者在长条件里容易读错。

3.8 数据流管道

管道把左侧的值作为右侧调用的输入，从而把嵌套调用写成自上而下的序列。

```
// 嵌套写法
println(to_string(len(trim(input))))

// 管道写法
input -> trim -> len -> to_string -> println
```

管道左侧必须是一个已声明的值，右侧是一个可调用目标。这一限制使管道与短闭包在语法上可以共存而不冲突：需要函数值时按下文所述的规则解释，需要求值时按管道解释。

3.9 短闭包

当上下文要求一个函数值时，形如 标识符 -> 表达式 的写法被解释为短闭包：标识符是参数名，表达式是函数体。参数类型与返回类型由上下文的函数类型给出。

```
var f: fn(i64) -> i64 = t -> t + 1
var g: fn(i64) -> i64 = n -> n * n

f = t -> t * 10          // 赋值位置同样适用
```

判据是上下文而不是符号表：声明位置的标注已经给出了函数类型，赋值位置的目标类型在语义分析时已知。两种情形都能确定地解释为短闭包，其余位置保持管道的含义不变。

如果确实要在函数值位置写管道，把中间结果先赋给变量再传递，或者用显式的函数名构造函数值。

3.10 解引用

反斜杠是解引用记号，作用于指针、引用与表句柄，取出其所指的值。它只在 `unsafe` 上下文中可用。

```
unsafe {
    var p = alloc(8)
    deref_write(p, as_u8(7))
    var v = as_i64(deref(p))
}
```

这一记号把「此处发生了指针访问」写在表达式里，使得审计内存访问时可以在文本层面搜索，而不必追踪函数调用的实现。

3.11 原地修改

双百分号是原地修改记号：它表示在操作数的存储位置上更新值，而不是先取出副本再写回。

```
var total = 0
total %% 1      // 等价于 total = total + 1
total %% 10     // 等价于 total = total + 10
```

这一记号的意义有两层。其一是书写简洁：对累加、计数、状态推进这类高频操作，它比复合赋值更短。其二是语义明确：编译器可以确定这是就地更新，无需先取副本，在表元素、结构字段这类位置上避免多余拷贝。

3.12 存在判定与安全默认值

问号后缀判定值是否存在；若存在则取出，否则走另一条路径。双问号是空值合并：左侧有值则取之，否则取右侧。

```
var port = config["port"] ?: 8080

if user? {
    println(user.name)
}
```

安全默认值的右操作数只在左侧缺失时求值，因此可以写有副作用或开销较大的表达式。

3.13 范围与切片

范围用两个点表示，左闭右开；三个字符的形式包含右端点。范围用于循环与切片。

```
for i in 0..10 { }    // 0 到 9
for i in 0..=10 { }   // 0 到 10
var head = xs[0..3]    // 前三个元素
```

3.14 类型判定

类型判定用于询问一个动态类型值的实际类型，配合分支使用，使分支内的取用是安全的。

```
var v: any = 42
if v is i64 {
    var n = as_i64(v)
}
```

判定与转换分开书写，而不是在判定中隐式完成转换。这样每一处的类型假设都是显式的，阅读时不必回推。

第 4 章 语句

语句描述控制流与状态的改变。本章逐一说明各类语句，并给出它们在书写上的一致做法。语句之间以自动分号分隔（见 1.10），因此不必为每行补分号，也不应依赖缩进决定结构。

4.1 变量声明

变量用 `var` 声明，常量用 `const` 声明。常量必须在声明处确定取值，此后不可重新绑定。

```
var count = 0
var name: string = "tie"
const MAX = 1024
```

作用域

变量在所在块内可见，块结束时消失。内层块可以声明与外层同名的变量，此时内层遮蔽外层；编译器对遮蔽给出提示，因为它常是笔误。

函数参数与函数体的局部变量处于同一作用域。闭包可以读取其外层函数的变量，规则见第 5 章。

4.2 赋值

赋值把右侧的值写入左侧的目标。复合赋值形式把运算与写入合并。

```
count = 10
count += 1      // 自增
count *= 2      // 倍增
count %= 5      // 原地取余并写回
```

赋值目标的类型与右侧表达式必须相容。语言不做隐式的窄化转换，需要窄化时显式书写转换。

4.3 条件分支

if 接受一个布尔表达式。else if 串接多个条件，else 收尾。条件必须严格是 bool——语言不接受用整数代表真伪，这排除了把赋值误写成比较的一类错误。

```
if score >= 90 {
    grade = "A"
} else if score >= 60 {
    grade = "B"
} else {
    grade = "C"
}
```

4.4 循环

循环有三种形态：条件循环、范围循环、集合循环。三者覆盖了绝大多数遍历需求，因此不需要通用的三段式循环语句。

```
while running {
    step()
}

for i in 0..10 {
    println(to_string(i))
}

for item in items {
    handle(item)
}
```

遍历文本使用码点迭代，这样多字节字符不会被切断：

```
for ch in text.chars() {
    // ch 是一个完整字符
}
```

循环标签

嵌套循环中，若需要从内层直接跳出到外层之外，用标签指明目标。标签写在循环语句之前。

```
outer: for a in 0..10 {
    for b in 0..10 {
        if b == 5 { break outer }
        if b == 2 { continue outer }
    }
}
```

无标签的 break 与 continue 作用于最近一层循环。带标签的跳转在编译期校验目标存在，标签未匹

配即报错。

4.5 多分支选择

switch 按值分派到若干分支。每个分支自动结束，不存在落入下一分支的情况，因此不需要 break。

```
switch code {  
  case 1, 2:  
    println("小")  
  case 3..7:  
    println("中")  
  default:  
    println("其他")  
}
```

分支可以匹配枚举并同时取出其携带的数据，此时编译器要求所有分支被覆盖，遗漏即报错。这是把「几种情况之一」的数据处理写完整的主要手段。

```
switch shape {  
  case Shape.Circle(r):  
    println(to_string(r))  
  case Shape.Rect(w, h):  
    println(to_string(w * h))  
}
```

4.6 跳转

return 结束当前函数并带回值；yield 在生成器中产生一个值并继续执行；panic 报告不可恢复的错误并终止当前执行流（见第 8 章）。

```
func pick(xs: table<i64>) -> i64 {  
  for x in xs {  
    if x > 0 { return x }  
  }  
  return 0  
}  
  
func counter(n: i64) -> table<i64> {  
  for i in 0..n {  
    yield i  
  }  
}
```

goto 在需要把多层嵌套的清理路径收敛到一处时使用。它只允许在同一个函数内跳转，且不允许跳入循环或分支的内部——这两条限制使它不会破坏块作用域。

```
if open_a() != 0 { goto cleanup_a }  
if open_b() != 0 { goto cleanup_b }  
// ...  
cleanup_b:  
  close_b()  
cleanup_a:  
  close_a()
```

4.7 表达式语句

一个表达式后面接语句结束，构成表达式语句。它用于调用有副作用的函数。

```
println("hello")
items -> sort -> print_all
```

没有副作用的表达式语句通常意味着书写疏漏，编译器会对此给出提示。

第 5 章 函数

函数是行为的单位。本章说明函数的定义与调用形态，并重点说明参数——参数是调用方与被调方之间的契约，写好参数规则，函数的行为边界就清楚了一半。

5.1 定义与调用

函数定义给出名称、参数与返回类型；返回类型不可省略。函数体是语句块，最后一条语句通常是 `return`。

```
func add(a: i64, b: i64) -> i64 {
    return a + b
}

var s = add(1, 2)
```

对外可见的函数标注 `pub`。可见性同时受模块与命名空间约束（见第 9 章）。

5.2 参数传递约定

参数的传递约定写在参数名的位置，共四种：按值、可写借用、只读借用、所有权转移。约定是声明的一部分，调用方不需要也无法改变它。

形态	语义
<code>x: T</code>	按值。被调方得到一份副本，改动不影响调用方。默认约定。
<code>ref x: T</code>	可写借用。被调方就地读写调用方的对象，改动对调用方可见。
<code>immu x: T</code>	只读借用。被调方只能读取，不能写入；用于避免大对象的复制开销。
<code>move x: T</code>	所有权转移。实参的所有权移交被调方，调用方此后不得再使用该变量。

```
func by_value(x: i64) -> i64 { return x }
func by_ref(ref t: table<i64>) -> i64 { table_push(t, 1); return len(t) }
func by_read(immu t: table<i64>) -> i64 { return len(t) }
func by_move(move t: table<i64>) -> i64 { return len(t) }
```

约定的互斥与组合

可写借用与只读借用互斥：同一个参数不能既允许写又声明只写。所有权转移与两者也互斥：所有权已经交出，无从借用。同一个修饰重复书写视为笔误，编译器报告错误。

四种约定是互斥的完备划分——每个参数恰好属于其中一种。这一设计使得阅读函数签名即可知道它对实参做了什么，不需要查看函数体。

可写借用的限制

可写借用要求实参是可寻址的对象（变量、字段、表元素），而不是临时值。这样「改动会写回」这一承诺才有着落。字面量与调用结果不能作为可写借用的实参。

所有权转移后的使用

实参被移出后，调用方再使用该变量会被检出。这条检查在移动语义检查阶段执行，覆盖直接使用与间接使用。

```
var data = table_new_i64()
table_push(data, 1)
var n = by_move(data)
println(to_string(len(data))) // 错误：变量已移出
```

5.3 默认参数

参数可以带默认值。带默认值的参数必须连续排在参数表末尾，因此调用方只需按顺序给出前面的实参。

```
func connect(host: string, port: i64 = 8080, timeout: i64 = 30) -> i64 {
    // ...
    return 0
}

connect("localhost")           // port 与 timeout 取默认
connect("localhost", 9090)     // timeout 取默认
connect("localhost", 9090, 60) // 全部给定
```

默认值在编译期确定，因此它必须是常量表达式——不含副作用，也不依赖调用点的作用域。这一限制使得默认值可以安全地在多个调用点之间共享。

可写借用与所有权转移的参数不带默认值：借用需要一个真实对象，所有权转移需要一个真实来源，两者都无法凭空构造。

5.4 命名实参

调用时可以按参数名指定实参。命名实参写在位置实参之后，可以乱序。这样做的好处是调用点自解释——不必回看函数定义即可知道每个值对应什么。

```
connect(host: "localhost", timeout: 60)
connect("localhost", timeout: 60)
```

命名实参可以跳过中间带默认值的参数。上例第二个调用中 port 未给出，取默认值。这一规则与默认参数「连续居尾」的约定配合，使得「只改其中一项」的调用写起来最短。

调用形式的三条规则：位置实参在前、命名实参在后；同一个参数不得被指定两次；命名不得与已被位置实参占用的参数冲突。违反任一条即报错，并指明具体位置。

与方法的一致性

方法调用（见第 6 章）的命名实参与默认参数规则与函数完全一致。接收者不参与命名——它由调用对象给出，也不能被默认值补齐。

5.5 变长参数

参数表末尾可声明变长参数，接收零个或多个同类型实参。变长参数不带默认值，也不能是借用。

```
func sum(rest: ...i64) -> i64 {
    var total = 0
    for x in rest {
        total += x
    }
    return total
}

sum()
sum(1, 2, 3)
```

调用时可用展开记号把一个表摊开成实参序列。

```
var xs: table<i64> = [1, 2, 3]
var t = sum(xs...)
```

5.6 返回值

函数可以返回多个值，返回类型写成元组。调用方解构接收。

```
func divmod(a: i64, b: i64) -> (q: i64, r: i64) {
    return a / b, a % b
}

var q, r = divmod(7, 3)
```

元组分量的名称是可选的，但推荐书写：它同时作为调用点的解构依据与文档。

5.7 一等函数

函数可以作为值传递、返回与存放。函数类型的写法是 `fn(参数类型...) -> 返回类型`。

```
func apply(g: fn(i64) -> i64, v: i64) -> i64 {
    return g(v)
}

func twice(g: fn(i64) -> i64, x: i64) -> i64 {
    return g(g(x))
}
```

具名函数可直接用作函数值，不需要额外的取地址或包装。

5.8 闭包

闭包是携带环境的函数值。它由函数字面量产生，可以读取其定义处的变量。

```
var base = 100
var bump: fn(i64) -> i64 = func(t) -> i64 { return t + base }
println(to_string(bump(1))) // 101
```

闭包字面量的参数类型在上下文给出函数类型时可以省略，由上下文补齐。没有上下文时报错，而不是猜测一个类型。

5.9 短闭包

单参数、单表达式的闭包可以写成短形式：参数名、箭头、表达式。

```
var f: fn(i64) -> i64 = t -> t + 1
var g: fn(i64) -> i64 = n -> n * n + 1
```

短闭包可用在声明位置（标注给出类型）与赋值位置（目标类型在语义分析时已知）。这两种位置都要求一个函数值，因此箭头两侧的解释是确定的。完整形态与短形态可以混用，二者语义相同。

5.10 捕获面

闭包能捕获什么，由捕获面白名单规定。这条规则的目的很具体：保证逃逸出定义处的闭包不会持有已经失效的引用。

可捕获	说明
外围的按值参数	捕获的是调用时刻的值快照
外围的只读借用参数	捕获的是只读快照；不可写回
外围的局部变量	捕获的是值快照
常量	常量本身，无副作用
全局变量	始终可见，不经捕获通道

可写借用参数不在白名单内，这是本章最需要记住的一条。原因不是书写风格，而是语义：可写借用不是副本，而是指向调用方对象的引用。若闭包捕获了它并逃逸出去，引用所指的对象可能已经不存在。

```
// 错误：闭包捕获可写借用参数
func make(ref t: table<i64>) -> fn() -> i64 {
    return func() -> i64 { return len(t) }
}
```

在不安全上下文中，这条限制放宽——那里由书写者承担引用有效性的责任（见第 11 章）。

捕获的语义

除可写借用之外，捕获一律是值语义：捕获发生在闭包创建的时刻，此后外围变量再改变，闭包看到的仍是创建时的值。表作为捕获对象时同样按值捕获，闭包内对表内容的修改不会影响外围的同名表。

这一组规则的组合效果是：闭包不产生共享可变状态，因此「多个闭包看到同一份数据的不同快照」这类问题在语言层面不会出现。跨线程传递函数值时，这一点尤其重要（见第 10 章）。

5.11 外部函数

extern 声明外部实现的函数，用于调用操作系统或第三方库。声明给出名称与签名，不含函数体。

```
extern func getenv(name: string) -> i64
```

外部函数可能返回指针或写入调用方的缓冲区，因此调用点通常需要不安全上下文。对系统调用与 C 库的封装应当集中在少数模块内，其余代码使用封装后的安全接口。

第 6 章 数据结构

tie 在结构设计上采取一条明确的分离原则：结构只描述数据，逻辑写在命名空间函数里，二者通过名称绑定。这条原则使「这是一个什么东西」与「可以对它做什么」成为两件独立的事，各自可以单独演进。

6.1 结构的定义

结构由字段组成，字段有类型与可选的默认值。结构是纯数据——不包含方法、不包含虚表、不引入隐式状态。

```
struct Point {  
    var x: f64 = 0.0  
    var y: f64 = 0.0  
    var label: string = ""  
}
```

字段的默认值使得构造可以只给出关心的部分，其余取默认。

6.2 构造与字段访问

构造按字段顺序给出实参。字段访问用点号。

```
var p = Point(1.0, 2.0)  
p.x = 3.0  
println(p.label)
```

结构作为参数时默认按值传递；对较大的结构使用只读借用以避免复制，需要修改时使用可写借用。

6.3 逻辑与数据分离

结构的行为写在同名命名空间内。命名空间函数的第一个参数是接收者。

```
namespace Point {  
    pub func dist(p: Point) -> f64 {  
        return sqrt(p.x * p.x + p.y * p.y)  
    }  
  
    pub func move_by(ref p: Point, dx: f64, dy: f64) {  
        p.x += dx  
        p.y += dy  
    }  
}
```

这一组织方式有一个直接好处：结构定义保持稳定，而逻辑可以增补、分组、按需导入。新增一个操作不需要改动结构所在的文件。

6.4 方法调用

点号调用是对命名空间函数的转发：编译器把 `obj.method(args)` 解释为「同名命名空间下的 `method`，接收者作为第一个参数」。

```
var p = Point(3.0, 4.0)
println(to_string(p.dist()))

p.move_by(1.0, 1.0)
```

转发意味着方法的参数规则与普通函数完全一致——默认参数、命名实参、借用与转移约定都可以用在方法上。接收者不参与命名，也不能被默认值补齐：它由调用对象给出。

```
struct Counter { var n: i64 = 0 }

namespace Counter {
    pub func bump(c: Counter, step: i64 = 1) -> i64 {
        return c.n + step
    }
}

c.bump()           // step 取默认
c.bump(step: 4)    // 命名实参
```

接收者的传递约定由命名空间函数声明决定：需要修改调用对象时把接收者声明为可写借用。

6.5 继承

结构可以继承另一个结构的字段，形成「更具体的类型」。继承只复用字段，不引入方法表——行为仍然由命名空间函数提供。

```
struct Shape {
    var name: string = ""
}

struct Circle extends Shape {
    var r: f64 = 0.0
}
```

子类型可以传给期望父类型的参数，字段按前缀布局，因此转换不需要额外工作。

6.6 语义限制

以下限制在编译期检查，它们共同保证结构是纯数据的这一性质。

- 结构体内不得定义函数；行为写在同名命名空间内
- 字段名在同一结构内唯一
- 继承链不得成环
- 字段默认值必须是常量表达式
- 结构作为参数时不得隐式发生窄化转换

这些限制不是风格偏好，而是设计后果：结构不携带行为，就没有虚表、没有构造顺序问题、没有隐式析构。调试时看到一个结构值，就看到了它的全部状态。

6.7 与元组的关系

元组适合表达「几个值临时组合」；结构适合表达有名字、会长期存在、有自己行为的概念。判据是命名与复用：需要按名字取用、需要在多处传递，就用结构。

```
// 临时组合：用元组
var lo, hi = min_max(xs)

// 长期概念：用结构
struct Interval { var lo: i64 = 0
                  var hi: i64 = 0 }
```

第 7 章 接口

接口描述一组能力：满足这组签名要求的类型，就可以被当作这个接口使用。接口是泛型约束与多态的基础，也是模块之间约定边界的主要手段。

7.1 声明

接口用 `port` 声明，其中列出方法签名。签名只写参数与返回类型，不含实现。

```
port Drawable {
    pub func draw(self, ctx: i64) -> string
    pub func bounds(self) -> i64
}
```

`self` 是接收者占位，实现时由具体类型替换。

7.2 实现

类型通过 `impl` 块声明它实现某个接口，并在其中给出各方法。接口要求的每个方法都必须实现，遗漏在编译期报告。

```
struct Button {
    var label: string = ""
    var w: i64 = 0
    var h: i64 = 0
}

impl Drawable for Button {
    pub func draw(self, ctx: i64) -> string {
        return "Button(" + self.label + ")"
    }
    pub func bounds(self) -> i64 {
        return self.w * self.h
    }
}
```

实现块中的 `self` 指向实现类型本身，因此实现体可以直接访问该类型的字段。

7.3 作为约束

泛型参数可以标注接口约束。约束在编译期检查：传入的类型必须实现了该接口，否则报错。

```
func render_all<T: Drawable>(items: table<T>, ctx: i64) {  
    for it in items {  
        it.draw(ctx)  
    }  
}
```

带约束的泛型在编译期为每个具体类型生成一份实现。因此调用没有间接跳转，也能被内联——接口在这里不带来运行期开销。

7.4 作为对象

接口也可以作为运行期的对象类型使用：把不同具体类型的值放入同一个接口类型的变量或表中，调用时按实际类型分派。

```
var items: table<Drawable> = table_new_Drawable()  
table_push(items, button)  
table_push(items, label)  
for it in items {  
    it.draw(0)  
}
```

这一形态在需要异质集合时使用，例如界面中混合放置不同控件。开销与任何动态分派方案相同，因此在不必要时优先使用泛型约束的形态。

两种形态共用同一份接口声明。选择哪一种取决于调用点是否需要异质集合：类型在编译期全部已知，用约束；需要把不同类型放进同一个容器，用对象。

7.5 接口的组合

接口可以要求另一个接口，从而在能力之上再叠能力。实现时必须同时满足全部要求。

```
port Focusable {  
    pub func focus(self) -> bool  
}  
  
port Control {  
    pub func draw(self, ctx: i64) -> string  
    pub func focus(self) -> bool  
}
```

组合让约束可以按能力分片：小接口易于实现也易于复用，大接口由小接口拼成，实现方仍只需提供具体方法。

第 8 章 错误处理

tie 把「可预期的失败」与「不可恢复的故障」分开处理。前者是返回值的一部分，调用方必须面对；后者终止执行并给出位置信息。这样区分的原因是二者的处理方式根本不同：前者需要分支决策，后者需要诊断信息。

8.1 可预期的失败

可能失败的函数把失败编码进返回类型。语言提供两个通用型别用于此目的：一个表示「有值或无值」，一个表示「成功值或错误值」。

```
func find(xs: table<i64>, target: i64) -> Option<i64> {
    for x in xs {
        if x == target { return Option.Some(x) }
    }
    return Option.None
}

func parse_port(s: string) -> Result<i64, string> {
    if str_len(s) == 0 { return Result.Err("空输入") }
    return Result.Ok(to_int(s))
}
```

8.2 传播

当函数无法就地处理失败、需要把它交给调用方时，用问号后缀传播：成功则取出值继续，失败则立即返回。

```
func config_port(cfg: map<i64>) -> Result<i64, string> {
    var raw = cfg["port"]?
    var port = parse_port(raw)?
    return Result.Ok(port)
}
```

传播记号把「错误向上传递」这条最常见路径写成一个字符，从而让函数体里的正常路径保持直线——不必为每一层调用写一段相同的分支。

8.3 分支处理

需要针对具体情况分别处理时，用多分支匹配展开结果类型。编译器要求覆盖全部情形。

```
switch parse_port(input) {
    case Result.Ok(v):
        println("端口 " + to_string(v))
    case Result.Err(msg):
        println("失败: " + msg)
}
```

对于「缺失则取默认」这一常见需求，用安全默认值记号直接表达，不必展开分支。

```
var port = cfg["port"] ?: 8080
```

8.4 标准组合子

标准库提供一组转换与组合函数，用于在结果类型之间衔接，减少手写分支。

```
var doubled = parse_port(s) -> map_result(v -> v * 2)
var len_ok = parse_port(s) -> and_then(validate_range)
```

8.5 不可恢复的故障

当故障意味着「程序状态已经不可信」时，用 `panic` 终止。它携带一条消息，用于定位问题。

```
if len(buf) == 0 {
    panic("缓冲区不应为空")
}
```

在明确划定边界的场合可以捕获 `panic` 并转为普通错误，例如服务的请求处理循环：单个请求的故障不应终止整个进程。捕获点应当少而集中，且必须能说明为什么在此处恢复是安全的。

内建检查

数组越界、除零、整数溢出（在带检查的运算中）会触发内建检查并报告位置。这些检查不可关闭，因为它们的成本相对故障后果可以忽略。

8.6 错误信息

错误值应当说明发生了什么、在哪一步发生。把上下文带在错误里，比在每一层重新描述问题更有用。

```
return Result.Err("读取配置失败：字段 " + name + " 缺失")
```

错误类型的选取遵循一条经验：能枚举的用枚举，不能枚举的用文本。当调用方需要按类别分支时用枚举；当调用方只是向上传递或打印时用文本。

第 9 章 模块与导入

程序由多个文件组成，文件之间通过导入建立依赖。本章说明导入的语法、可见性规则与命名空间的角色。

9.1 导入

导入把另一个文件的内容引入当前文件，并可选地给它一个别名。别名用于限定访问，使来源在调用点清晰可见。

```
import "./geometry.tie" as geo
import "./format.tie"

var d = geo.distance(a, b)
```

不写别名时，被导入文件的公开名称直接进入当前作用域，适合导入内容很少的文件。来源较多时使用别名，避免名称冲突。

9.2 命名空间

命名空间把一组函数组织在同一个名称之下。命名空间可以跨越多个文件：若干文件声明同一个命名空间时，它们的内容合并为一份。

```
namespace math {
    pub func gcd(a: i64, b: i64) -> i64 {
        if b == 0 { return a }
        return gcd(b, a % b)
    }
}
```

命名空间可以嵌套，形成层次化的名称。嵌套的层级同时表达归属关系：内层命名空间只在外层可见的范围内被引用。

跨文件的分片需要由主文件显式导入分片，命名空间才会合并。只在别处导入分片而主文件未导入它，会得到「名称未定义」——这是导入关系的正常结果，不是缺陷。

9.3 可见性

声明默认是私有的，只有标注 `pub` 才对外可见。可见性在导入边界上检查：被导入文件的私有名称不出现在导入方的作用域中。

```
func helper() { }          // 仅本文件可见
pub func api() { }         // 对外可见
```

私有是本语言在处理跨文件访问时的默认选择。理由很实际：默认私有使得一个文件内部的调整不会意外影响其他文件，公开面需要显式付出决策成本。

9.4 标准库

标准库以模块为单位提供。常用模块与其职责如下。

模块	职责
string	文本查找、切分、替换、大小写
utf	码点与字节的转换、字符分类
collection	表与映射的常用操作
sort	排序与查找
math	数值函数与统计
regex	正则匹配与替换
fs	文件与目录
path	路径拼接与分解
json	数据交换格式的读写
time	时间与日期
random	随机数
process	进程与环境
net	网络通信
crypto	散列与加密

9.5 依赖与版本

包管理器按版本解析依赖，并把解析结果与校验信息记录在项目文件中。依赖的引入应当在项目级完成，而不是在源文件中按路径直接引用。

```
// 项目文件中记录依赖与版本约束
tie.pkg
  name: my-project
  require: json ^2.1, net ~1.4
```

版本约束使用语义化的范围记号，使得依赖升级的边界在文件里可见。

第 10 章 并发

并发在 tie 里由两类构造承担：actor 提供隔离的状态单元，任务图提供带依赖的并行调度。两者的共同前提是「默认不共享可变状态」——共享是通过显式通道或消息传递发生的，不是通过共享内存偶然发生的。

10.1 并发的基本立场

数据竞争通常来自两处：两个执行流访问同一块可写内存；或者一处持有指向另一处栈帧的引用。语言在设计上分别封堵：

- 共享可变状态通过消息传递表达，通道有明确的所有权归属
- 可写借用不能被闭包捕获，因此不会随函数值逃逸到别的执行流
- 跨线程传递的值按值复制，不建立隐式别名

10.2 状态单元

actor 是并发的基本单位：它拥有自己的状态，只通过消息与外界交互。对它的状态的访问全部发生在它自己的执行流里，因此不需要锁。

```
actor Counter {
    var n: i64 = 0

    func inc(step: i64) {
        n += step
    }

    func value() -> i64 {
        return n
    }
}

var c = Counter()
c.inc(5)
println(to_string(c.value()))
```

调用 actor 方法即发送消息。消息在队列中排队，按到达顺序处理，因此状态变化的顺序是可预期的。

10.3 消息与返回

消息可以携带数据。需要结果时，调用等待回复并把结果作为表达式的值。带返回值的调用与普通调用写法相同，等待是隐式的。

```
var total = c.value()
```

发送方与接收方之间传递的数据按值语义复制或转移，因此不存在双方同时持有同一块可写内存的情形。

10.4 任务与依赖

当工作之间存在依赖关系时，用任务图描述：节点是工作单元，边是依赖。调度器按依赖顺序推进，互不依赖的节点并行执行。

```
var g = graph()
var a = g.add(fetch_a)
var b = g.add(fetch_b)
var c = g.add(merge, a, b) // c 依赖 a 与 b
g.run()
```

任务图中传递的值遵循与 actor 相同的规则：跨节点传递即复制或转移。这使并行部分与串行部分可以自由混合，而不必为边界处的数据归属单独设计。

并发构造的选择依据是「状态是否需要长期存在」。需要长期存在并被多方交互的状态放在 actor 里；一次性的并行计算用任务图。

10.5 与闭包捕获的关系

第 5 章的捕获面白名单在这里体现为一条直接的安全保证：由于可写借用不可被闭包捕获，任何函数值都不持有对他人栈帧的可写引用。把函数值交给别的执行流时，不可能带入一个可写别名。

这条规则也解释了为什么跨线程传递闭包是安全的：闭包携带的全部是被捕获时刻的值快照，每个执行流拿到的是各自独立的副本。

第 11 章 不安全机制

语言的安全保证建立在「不出现未定义行为」之上。绝大多数代码不需要违反这条前提，因此默认落在安全区内；少数需要直接操作内存、调用外部接口或放宽检查的场合，语言提供一处显式标记：不安全区。

11.1 安全区的边界

安全区内不允许出现以下构造——它们在编译期被拒绝，而不是在运行期碰运气。

- 指针类型作为变量、参数或字段类型
- 对指针的解引用
- 指针与整数之间的转换
- 调用标记为不安全的函数
- 闭包捕获可写借用参数

这些限制覆盖了内存不安全的主要来源：悬垂引用、越界访问、类型混淆、数据竞争。

11.2 不安全区

不安全区有两种形态：标记为不安全的函数，以及函数体内的不安全块。二者都把「此处由书写者承担内存正确性的责任」写在代码里。

```
unsafe {
    var buf = alloc(16)
    deref_write(buf, as_u8(1))
    var v = as_i64(deref(buf))
    free(buf)
}
```

不安全函数在签名上标注 `unsafe`。安全代码调用不安全函数必须自己处在不安全区内，这一要求由编译器强制：调用链上每一处跨越都必须显式。

11.3 指针操作

指针类型在类型名上体现为 `ptr` 与元素类型。指针的分配、读写、偏移与释放都在不安全区内进行。

```

var p = alloc(64)           // 分配
var q = ptr_add(p, 8)       // 偏移到第 8 字节
deref_write(q, as_i64(42))  // 写入
var n = as_i64(deref(q))    // 读出
var a = ptr_to_int(p)       // 转为整数（传给外部接口）
free(p)                     // 释放

```

指针与整数的双向转换使外部接口的调用可以表达：把地址作为整数传给不接受指针类型的接口，或把返回的整数重新解释为指针。

11.4 放宽的检查

不安全区内的部分检查放宽，由书作者承担后果。放宽的范围是明确的、有限的：指针相关的检查与借用逃逸的检查。其余检查——类型相容、参数个数、分支完备性——在任何地方都不会关闭，因为它们与内存布局无关。

闭包捕获可写借用的限制在不安全区内解除，用于需要把借用显式传递到回调的场合。此时借用的有效期由书作者保证。

11.5 审计友好

不安全机制的形态经过取舍：它不采用「关闭全文件检查」的开关，也不采用隐式的宽松规则，而是要求把每个不安全操作写在标记区内。这样做的目的是让审计可以按文本搜索完成。

- 搜索 `unsafe` 即可定位全部需要人工复核的位置
- 不安全区的边界在语法上可见，不依赖约定
- 解引用记号使指针访问在语句层面可辨认

封装外部接口时，把不安全操作收敛到少数模块，对外只暴露签名安全、行为可预期的接口。这样不安全区的总量可控，复用这些封装的使用方无需关心内部细节。

第 12 章 宏与元编程

宏让代码可以生成代码。`tie` 的宏在编译期执行，输入与输出都是代码片段——代码在这里被当作普通的值来传递，因此不需要为模板另造一套语言。

12.1 代码片段

`code` 是编译期类型，它的值是一段语法结构，而不是运行期的数据。`code` 值只能出现在宏相关的上下文里，不能存进变量跨阶段使用。

```

func twice(x: code) -> code {
  return quote {
    @x + @x
  }
}

```

12.2 宏的定义

宏用 `macro` 声明。宏在编译期被调用，其返回值替换调用点。由于展开发生在编译期，宏不带来运行期开销。

```
macro unless(cond: code, body: code) -> code {
  return quote {
    if !@cond {
      @body
    }
  }
}

unless(done, { cleanup() })
```

12.3 引用与插值

引用记号把一段代码包成 `code` 值而不立即求值；插值记号把 `code` 值嵌入到引用中。二者配合，使宏作者可以描述「带空洞的代码模板」。

```
quote { if @cond { @body } } // @cond 与 @body 是空洞
```

插值遵循与字符串插值一致的直觉：看到记号就知道此处会被替换，其余部分是字面结构。

12.4 编译期计算

常量表达式在编译期求值。这使得查表、派生量、配置校验等可以在产物生成之前完成，运行期只留下结果。

```
const TABLE_SIZE = 1 << 10
const MASK = TABLE_SIZE - 1

macro assert_const(cond: code) -> code {
  // 在编译期检查条件，不满足则报错
  return quote { }
}
```

12.5 使用边界

宏适合表达那些「写法重复但语义简单」的结构：断言、日志、资源收尾、派生实现。它们不适合承载主要业务逻辑，原因是宏展开后的代码在调试器里的位置信息不如直接书写的代码直观。

- 用宏消除重复的样板，不用宏制造新的语言
- 宏的展开结果应当可以用手写代码等价表达
- 宏的报错信息应当指向调用点，而不是展开内部

标准库的断言、日志与测试宏都遵循同一形态：调用点简洁，展开结果平凡，报错指向调用处。

第 13 章 所有权与移动

每个值都有明确的归属。绝大多数时候归属是显然的——值属于声明它的那个变量。当值需要在变量之间转手时，语言要求这件事被写出来，而不是悄悄发生。

13.1 默认按值

赋值与传参默认按值：新位置得到一份独立的值，原位置继续有效。两个变量此后互不影响，读写其中

一方不会改变另一方。

```
var a: i64 = 1
var b = a
b = 2
println(to_string(a)) // 1
```

13.2 显式移动

当不需要保留原变量、且希望避免复制开销时，用移动记号把所有权转出。此后原变量处于「已移动」状态。

```
var big = collect()
var owned = big // 复制或转移，按类型语义
consume(owned)
```

参数声明为 `move` 时，调用即转移：形参约定本身就表达了「这个值交给被调方」，调用方不必在调用点重写记号。

```
func take(move t: table<i64>) -> i64 { return len(t) }

var xs = table_new_i64()
table_push(xs, 1)
var n = take(xs)
// 此后 xs 不可再用
```

13.3 已移动状态

变量被移出之后再次使用会被检出。这条检查覆盖直接读取、再次传入、以及作为表达式的一部分使用。

```
var n = take(xs)
println(to_string(len(xs))) // 错误：变量已移出
```

检出发生在编译期，因此这类错误不会在运行期以「读到已释放对象」的形式出现。需要保留原值继续使用时不使用移动，改用按值或借用。

13.4 借用与转移的取舍

三种做法对应三种需求，选择依据是「调用方还需要这个值吗」。

需求	做法
调用方不需要继续用	声明 <code>move</code> ，把所有权交给被调方
调用方继续用，只读取	声明 <code>immut</code> ，避免复制
调用方继续用，且要被调方修改	声明 <code>ref</code> ，写回原对象
双方各持一份，互不影响	按值，接受一次复制

13.5 复制与克隆

按值传递对小的标量代价为零；对表这类动态结构则是一次真实的数据复制。需要明确表达「我要一个

独立的副本」时使用克隆，使复制的成本在代码里可见。

```
var snapshot = xs.clone()
```

可借用不产生副本，因此能修改原对象；但它有生命周期限制——借用不能被闭包捕获带走（见第 5 章），也无法越过函数边界长期存在。这组限制共同保证借用指向的对象始终有效。

第 14 章 界面元素块与图语法

有两类结构，用函数调用写出来会显著失去可读性：嵌套很深的界面元素树，以及需要表达拓扑关系的图。语言为二者提供专门的记号，使代码的形态接近它描述的结构本身。

14.1 元素块的写法

元素块以 @ 记号引入，块内书写元素与属性。元素之间的嵌套关系由书写的层次表达，不需要括号配对。

```
@ window "设置" {
  size: 480 x 320
  @ panel {
    @ label "服务地址"
    @ input host: "localhost"
    @ row {
      @ button "保存" on_click: save
      @ button "取消" on_click: cancel
    }
  }
}
```

这段写法与它描述的界面同构：外层窗口包含一个面板，面板包含一行文本、一个输入框与一个按钮行。改结构时改动的位置与视觉上的位置一致。

14.2 属性与绑定

元素的属性写在元素之后，形式为名称与值。值可以是字面量、变量或表达式；写表达式时用花括号界定，与字符串插值同一形态。

```
@ label title
@ progress value: done * 100 / total
@ input host: config.host
```

属性可以分行书写，也可以放在同一行。分行时每个属性独占一行，便于对齐与增删。

14.3 事件

事件用 on_事件名 的形式绑定到一个函数值。绑定接受具名函数，也接受短闭包。

```
@ button "重试" on_click: retry
@ button "清空" on_click: t -> clear_all()
@ input host: "localhost" on_change: host_changed
```

事件处理函数是普通函数，参数与返回类型由事件类型决定。由于函数值遵循第 5 章的捕获规则，事件处理只携带值快照，不持有对界面外部状态的可写引用。

14.4 数据与界面

界面元素可以引用所在作用域的值。值的更新触发界面刷新，刷新范围由引用关系决定——读取了某个值的元素才会重建，其余部分保持不动。

```
func view(state: AppState) -> i64 {
  @ window "计数器" {
    @ label "计数: " + to_string(state.n)
    @ button "加一" on_click: bump
  }
  return 0
}
```

14.5 图语法

图语法用于描述节点与它们之间的连接。节点声明与边声明分开书写：先给节点起名，再用箭头描述连接方向。

```
graph Pipeline {
  node fetch
  node parse
  node store

  fetch -> parse
  parse -> store
  store -> fetch
}
```

箭头方向即数据或控制流的流向。需要给边附加条件或权重时，在箭头之后书写。

```
parse -> store when valid
parse -> fetch when invalid
```

图作为值可以构造、组合与执行：构造产生图对象，组合把两个图按同名节点连接起来，执行按拓扑顺序推进。由于图是不可变的，组合不会修改原图，而是产生新图——这使得图可以在多个调用点之间安全共享。

元素块与图语法都不是独立语言，而是同一套表达式的书写形态。二者内部可以出现普通表达式、函数调用与闭包，也可以被普通代码构造出来。

第 15 章 工具链与语言服务

语言规范定义了什么是正确的；工具链让书写、检查与维护这件事变得可行。本章说明编译、诊断、格式化与文档生成的行为约定。

15.1 编译流程

编译分为若干阶段，每阶段有明确的输入输出，可以对中间产物单独检查。

阶段	职责
预处理	处理头部声明、导入关系与编译期开关
词法与语法	切分记号、构建语法结构，报告语法错误
语义分析	类型检查、约定检查、所有权与移动检查、捕获面检查
中间表示	下沉为与机器无关的中间代码
优化	常量折叠、内联、单态化展开、无用代码消除
生成	产出目标代码并与运行时链接

语义分析阶段的检查顺序是确定的：类型在前，约定居中，所有权与捕获在后。固定的顺序使得同一次编译里的报错不会因为检查次序而变化。

15.2 命令行

编译器的常用调用形态是一致而可组合的：源文件、输出、开关。

```

tiec app.tie          // 编译为可执行文件
tiec lib.tie -o lib.a // 编译为静态库
tiec app.tie --emit-ir // 只产出中间表示
tiec app.tie --check  // 只做检查，不产物
tiec app.tie --emit-pdf // 随产物输出结构报告

```

优化级别与目标平台属于命令行开关，不写在源文件里。这样同一份源码可以在不同目标下编译，而不必为每个目标维护一份文件。

15.3 诊断信息

每条诊断有稳定的编号、明确的类别与指向具体位置的坐标。编号使得错误可以被引用、被加入回归语料，而不必复述整段消息。

```

error[E00387] main.tie:12:5: 命名实参调用 'connect' 缺少第 2 个形参 'port' 的值
warning[W00008] main.tie:3:5: 变量 'tmp' 未被使用

```

诊断的撰写遵循三条约定：指出问题所在的位置；说明期望与实际的差别；在能给出修法时给出修法。

15.4 格式化与静态检查

格式化工具按统一规则重排代码，使风格差异不进入评审。它的输出是确定的：同一份代码格式化两次结果相同。

静态检查工具在编译之外给出建议，覆盖可能的疏漏——未使用的变量、被遮蔽的名称、恒真的条件、可以简化成管道表达式的嵌套调用。

```

tiefmt src/          // 就地格式化
tielint src/         // 静态检查

```

15.5 文档生成

以三条斜线书写的文档注释被工具提取，与声明一起生成 API 文档。文档注释支持与正文相同的排版记号，因此说明与示例可以写在一起。

```
/// 求两数的最大公约数。
///
/// 示例：
///      gcd(12, 18)    // 6
pub func gcd(a: i64, b: i64) -> i64 { }
```

15.6 交互式环境

交互式环境接受表达式并立即求值，用于试验标准库函数与语言构造。它与编译使用同一套语义实现，因此交互环境里的结果与编译产物的行为一致。

```
$ tie
> var xs = [3, 1, 2]
> xs -> sort
[1, 2, 3]
```

15.7 语言服务

语言服务为编辑器提供补全、跳转、悬停与实时诊断。它按文件增量重算，不要求整工程重新编译；诊断的编号与命令行输出一致，因此编辑器中的提示与构建日志可以直接对照。

工具链与语言本身共用同一份实现：解析、类型检查与诊断由同一套代码提供，这保证编辑器里看到的结论与编译结果不会出现分歧。

第 16 章 速查表

本章把全书的形态汇总成查阅表，供书写时快速定位。语义细节见对应章节。

16.1 文件与角色

```
type tie           // 泛型入口 (Type 角色)
type tie<logic>    // 逻辑代码 (默认)
type tie<class>    // 类库
type tie<script>   // 脚本
type tie<data>     // 纯数据
type tie<ui>       // 界面
type tie<db>       // 数据库
type tie<port>     // 接口
type tie<ir>       // 中间表示
```

16.2 声明形态

```

var x = 1           // 变量
const K = 10        // 常量
func f(a: i64) -> i64 { } // 函数
pub func g() { }     // 公开函数
struct S { var n: i64 = 0 } // 结构
enum E { A(x: i64) } // 枚举
namespace ns { }     // 命名空间
port P { pub func m(self) } // 接口
impl P for S { }     // 实现
actor A { var n: i64 = 0 } // 状态单元
macro m(x: code) -> code { } // 宏
import "./m.tie" as m // 导入
extern func e(a: i64) -> i64 // 外部函数

```

16.3 参数约定

```

func f(a: i64)           // 按值
func f(ref t: table<i64>) // 可写借用
func f(immud t: table<i64>) // 只读借用
func f(move t: table<i64>) // 所有权转移
func f(a: i64, b: i64 = 10) // 默认值（须连续居尾）
func f(a: i64, b: i64, c: i64) // 调用：f(a: 1, c: 3) 命名并跳过 b
func f(rest: ... i64)       // 变长参数

```

16.4 表达式记号

```

a -> f           // 数据流管道
t -> t + 1       // 短闭包（要求函数值的位置）
x in m           // 成员判定
x not in m       // 成员取反
v ?: 10          // 安全默认值
v?              // 存在判定
p\              // 解引用
n %% 1           // 原地修改
a ** b           // 幂
a **? b          // 带检查的幂
0..10    0..=10 // 范围

```

16.5 语句形态

```

if cond { } else { }
while cond { }
for i in 0..10 { }
for x in xs { }
for c in s.chars() { }
lbl: for i in 0..10 { break lbl }
switch v { case 1: }
return v
yield v
break    continue
goto cleanup
panic("msg")

```

16.6 常用内建

```

println(s)          // 输出一行
to_string(v)         // 转文本
str_len(s)           // 码点数
len(t)               // 元素数或字节数
str_char(s, i)       // 取第 i 个码点
char_code(c)         // 取码点的数值
table_new_i64()       // 新建动态表
table_push(t, v)      // 追加
map_new()             // 新建映射
file_read(p) file_write(p, s)
byte_read(p) byte_write(p, t) // 二进制安全
load_library(n) get_proc(h, n) dyn_call(p, ...)

```

16.7 诊断编号

诊断编号按类别分段，便于按区间检索。

区间	类别
E00000 – E00099	语法与词法
E00100 – E00199	内建函数调用形态
E00200 – E00399	类型与调用匹配
E00400 – E00499	导入、声明与作用域
E00500 – E00699	借用、移动与捕获
E00700 及以上	并发、宏与不安全区
W00001 – W00099	警告：可疑但合法

编号稳定是工具链的前提：编辑器、CI 与回归语料都按编号引用诊断，重排编号会同时破坏这三者。因此新增诊断一律取新号，不复用旧号。

16.8 设计要点回顾

本书的规则可以收敛成若干条彼此呼应的立场，它们共同构成语言的性格。

- 参数约定写在声明处，四种划分互斥且完备——读签名即知实参的去向
- 命名实参可跳过带默认值的参数，使「只改一处」的调用最短
- 闭包捕获除可写借用外一律按值，因此不产生共享可变状态
- 并发通过消息与任务图表达，共享不通过共享内存偶然发生
- 不安全操作写在标记区内，审计可按文本搜索完成
- 常见的书写意图有专有记号：管道、短闭包、成员判定、原地修改、安全默认值